Introducción a Ciencia de Datos

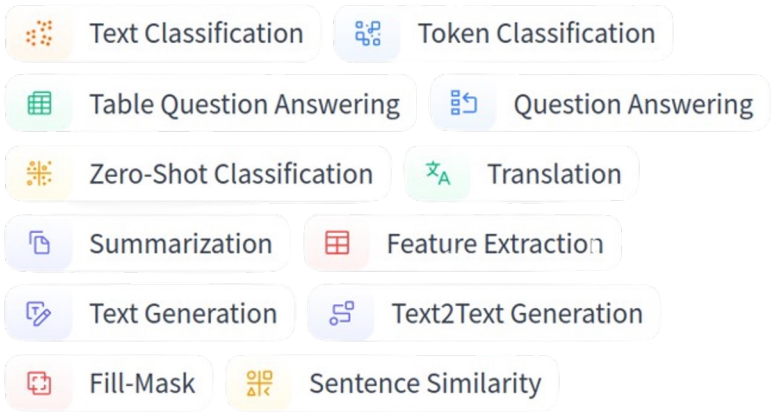
Natural Language Processing

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Natural Language Processing

¿Que es en el NLP?

Modelos capaces de entender el lenguaje humano



Enfoque “Tradicional”:

Para cada problema específico existen algoritmos especializados:

- **Basados en reglas** lingüísticas
- **Basados en datos** con cientos de miles de modelos

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

¿Cómo procesamos texto?

Diversas técnicas:

1. Bag of Words
2. *TF - IDF*
3. **Embeddings**



Bag of Words

Conjunto de las diferentes palabras que aparecen en un documento asociadas a la cantidad de veces que aparecen en el documento.

Consiste en:

- Armado de vocabulario
- Conteo de palabras en vector

Documento!

Ejemplo:

Todos los estados tienen sus propias leyes. Todos los países tienen sus propias culturas

```
V = {Todos: 2, los: 2, estados: 1, tienen: 2, sus: 2,
propias: 2, leyes: 1, paises: 1, culturas: 1}
```



Bag of Words

Ejemplo:

D1: Todos los estados tienen sus propias leyes.
D2: Todos los países tienen sus propias culturas

	todos	culturas	estados	paises	sus	los	leyes	tienen	propias
D1	1	0	**1**	0	1	1	1	1	1
D2	1	1	0	1	1	1	0	1	1
sum	2	1	1	1	2	2	1	2	2

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Tokenización

Tokenización es el proceso de convertir un texto en *tokens* que serán el input de nuestros modelos. Usualmente este proceso está acompañado de otros, como remoción de stop words, stemming, remoción de puntuación, n-gramas, etc

Ej:
"Como me gusta mucho Python, estoy usando Python todos los días"

T1: como|me|gusta|mucho|python|estoy|usando|python|todos|los|días

T2: como|me|gus|ta|mucho|pyth|on|estoy|usando|pyth|on|todos|los|días

Tipos de tokenización

No todos los modelos "leen" de la misma forma. La elección del tokenizador define qué tan bien el modelo entiende el vocabulario.

Tokenización por Palabras

- Separa el texto basándose en espacios o puntuación. Muy intuitivo para humanos. Pero, el vocabulario se vuelve gigante.

Tokenización por Caracteres

- Trata a cada letra como un token. Vocabulario minúsculo. Nunca falla con palabras nuevas. Las secuencias se vuelven larguísimas y se pierde el significado semántico de la palabra completa.

Tokenización por Sub-palabras

- Descompone palabras frecuentes en unidades completas y palabras raras en fragmentos con significado. Estándar actual. Permite que el modelo entienda que "corriendo" y "correr" comparten la raíz "corr", ahorrando memoria y mejorando la comprensión.

Para los interesados, leer sobre *Byte Pair Encoding*.

Stopwords y puntuación

Antes de que el modelo analice el significado, debemos quitar lo que no aporta valor semántico.

- **Stopwords:** Palabras muy frecuentes (artículos, preposiciones) que actúan como "pegamento" pero no definen el tema (ej: "el", "de", "y").
- **Puntuación:** Los signos como ?, !, . suelen tratarse como tokens separados o eliminarse para evitar ruido.

```
from nltk.corpus import stopwords
import string

text = "Los programadores están programando programas en Python"
tokens = text.lower().split()
stop_words = set(stopwords.words('spanish'))
filtered = [w for w in tokens if w not in stop_words and w not in string.punctuation]
```

```
[programadores, están, programando, programas, Python]
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Stemming

Técnica que busca reducir la palabra a su raíz. Corta el final de la palabra siguiendo reglas heurísticas. Es rápido pero puede generar palabras que no existen.

- **Corriendo, Corrió, Correrá** *-> corr*

```
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
import string

text = "Los programadores están programando programas en Python"
tokens = text.lower().split()
stop_words = set(stopwords.words('spanish'))
filtered = [w for w in tokens if w not in stop_words and w not in string.punctuation]
ps = PorterStemmer()
stems = [ps.stem(w) for w in filtered]
```

```
[program, est, program, program, Python]
```

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Lemming

Como el stemming, busca reducir la palabra a su raíz. En este caso, busca la raíz lingüística (lema) usando un diccionario. Es más preciso pero requiere más cómputo.

- **Corriendo, Corrió, Correrá** -> *correr* (verbo en infinitivo)
- **Mejor, Buenos** -> *bueno* (adjetivo base)

```
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
import string

text = "Los programadores están programando programas en Python"
tokens = text.lower().split()
stop_words = set(stopwords.words('spanish'))
filtered = [w for w in tokens if w not in stop_words and w not in string.punctuation]
# La lematización en NLTK suele requerir el idioma inglés o Spacy para español
lemmatizer = WordNetLemmatizer()
lemas = [lemmatizer.lemmatize(w) for w in filtered]
```

```
[programador, estar, programar, programa, Python]
```

N Gramas

No siempre las palabras sueltas tienen sentido. Los **N-gramas** capturan el contexto agrupando tokens adyacentes. Ayudan a capturar **frases hechas** o **conceptos compuestos** (ej: "San Francisco" o "Máquina de escribir").

- **Unigrama (n=1):** ["Me", "gusta", "Python"]
- **Bigrama (n=2):** ["Me gusta", "gusta Python"]
- **Trigrama (n=3):** ["Me gusta Python"]

```
from nltk.corpus import stopwords
from nltk.util import ngrams
import string

text = "Los programadores están programando programas en Python"
tokens = text.lower().split()
stop_words = set(stopwords.words('spanish'))
filtered = [w for w in tokens if w not in stop_words and w not in string.punctuation]
bigrams = list(ngrams(tokens, 2))
```

```
[programadores están, están programando, programando programas, programas Python]
```

TF – IDF

Set de documentos

Métrica de *information retrieval* usada para calcular el **peso** de una palabra dentro del *corpus* de documentos.

De las siglas **T**erm **F**requency - **I**nverse **D**ocument **F**requency

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

Donde

TF:
Cantidad de ocurrencias del término *t* en el

$$TF(t,d) = \sum_{x \in d} fr(x,t)$$

IDF:
Inversa de la cantidad de documentos donde aparecce el término *t*

$$IDF(t) = log \frac{|D|}{1 + |\{d / t \in d\}|}$$

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Embedding Models

Queremos obtener nuevamente una representación vectorial del texto... Convertimos cada **token** a un vector usando una red neuronal

El conocimiento **necesario** para hacer esto excede al curso y podemos discutirlo en otro momento.



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Embedding Models

Normalmente basados en estructuras **Encoder - Decoder** (Deep Learning)

Ejemplo con Bag of Words:

	todos
D1	1
D2	1
D3	0
D4	1
…	…
D1000	1

Comprime

Encoder

todos
0.2334
-0.433
0.983

Descomprime

Decoder

	todos
D1	1
D2	1
D3	0
D4	1
…	…
D1000	1

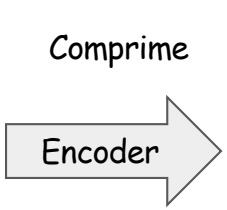
UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Embedding Models

Normalmente basados en estructuras **Encoder - Decoder** (Deep Learning)

Ejemplo con Bag of Words:

	todos
D1	1
D2	1
D3	0
D4	1
…	…
D1000	1



todos
0.2334
-0.433
0.983

Una vez entrenado el **Encoder - Decoder**, tiro el Decoder!

Y conseguí una representación con menor cantidad de dimensiones de la palabra

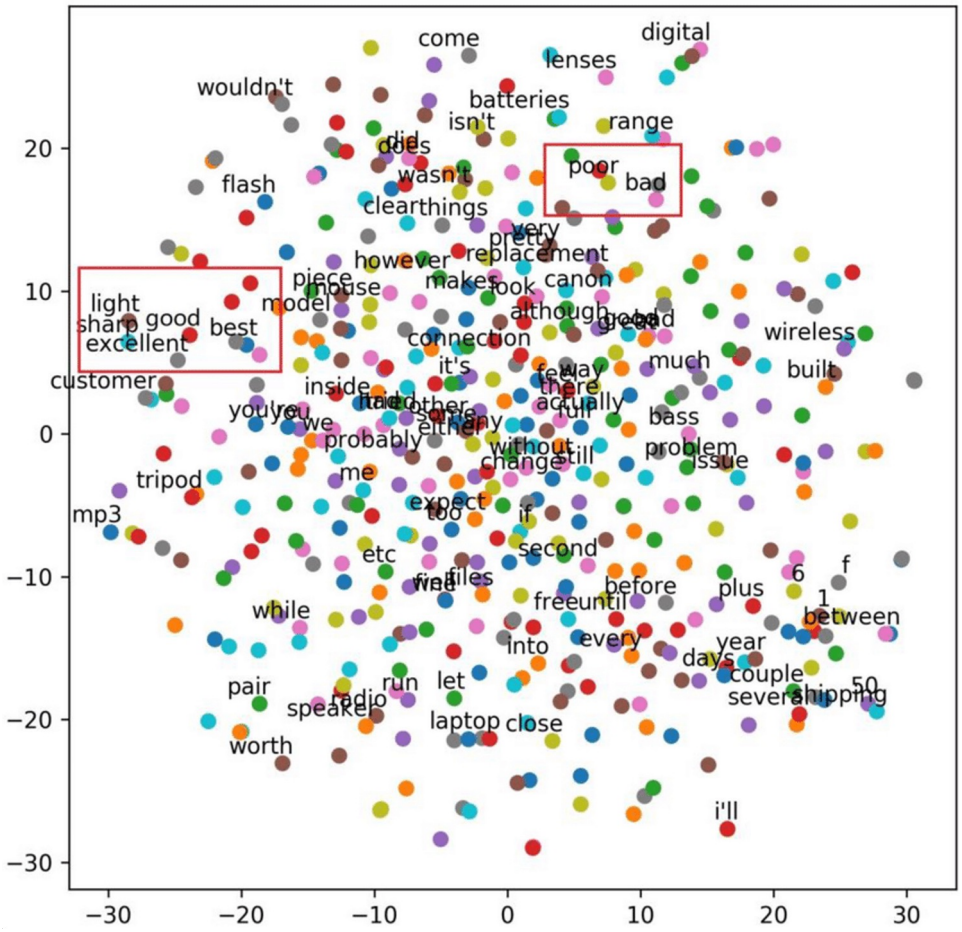
UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Texto a vectores

Implica una representación espacial del texto!

Algebra de vectores:

- *Qué tan parecida es una palabra a otra*
- *Qué tan cerca está una de la otra*

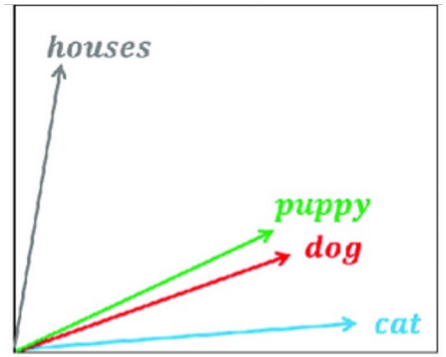


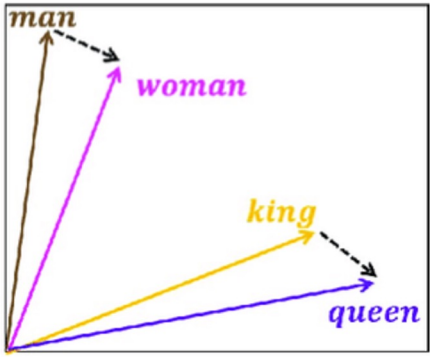
Texto a vector?

Realmente hablo de algebra

Si “roto” una palabra en mi espacio vectorial, conceptos similares “rotan” de manera similar

	d1	d2	d3	d4	d5	d6	d7
dog →	0.6	0.9	0.1	0.4	−0.7	−0.3	−0.2
puppy →	0.5	0.8	−0.1	0.2	−0.6	−0.5	−0.1
cat →	0.7	−0.1	0.4	0.3	−0.4	−0.1	−0.3
houses →	−0.8	−0.4	−0.5	0.1	−0.9	0.3	0.8
man →	0.6	−0.2	0.8	0.9	−0.1	−0.9	−0.7
woman →	0.7	0.3	0.9	−0.7	0.1	−0.5	−0.4
king →	0.5	−0.4	0.7	0.8	0.9	−0.7	−0.6
queen →	0.8	−0.1	0.8	−0.9	0.8	−0.5	−0.9



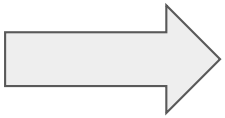


UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Embedding Models

Todos los estados tienen sus propias leyes. Todos los países tienen sus propias culturas

Cómo?



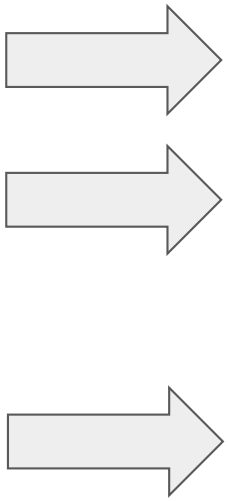
0.22	-0.1	**0.43**	…	0.13	0.15

Todos

los

…

culturas



0.1	-0.15	0.22	…	0.1	0.1
0.1	-0.15	0.22	…	0.1	0.1
0.1	-0.15	0.22	…	0.1	0.1

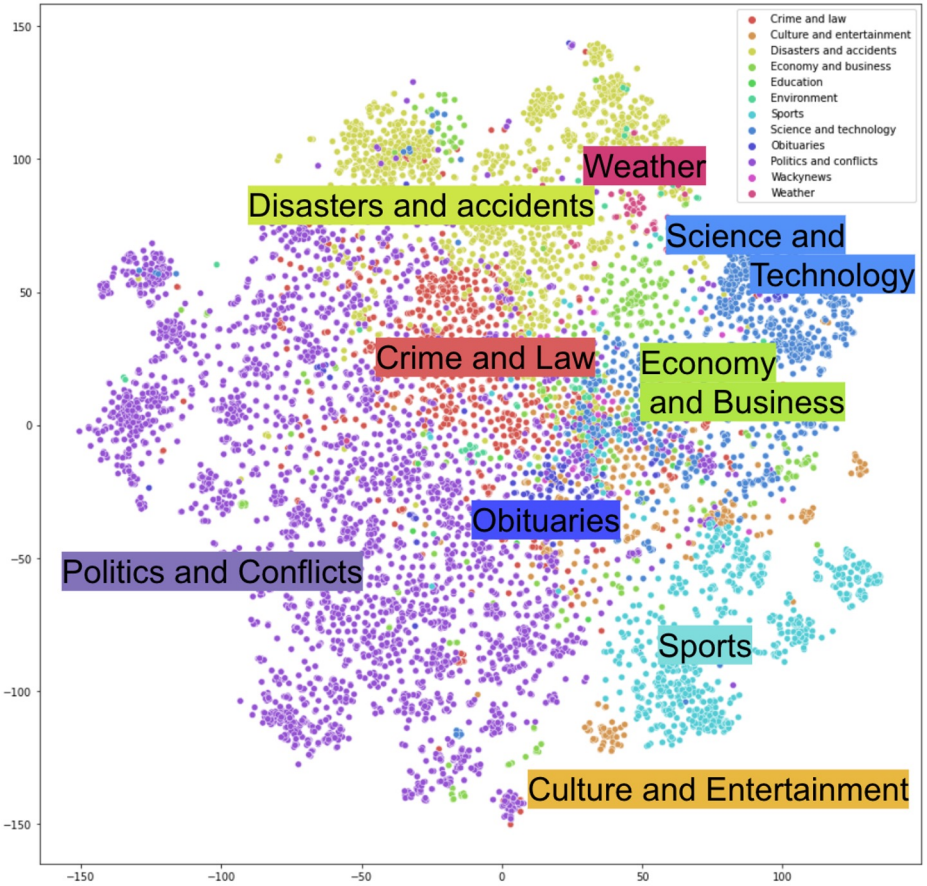
Promedio por componente

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Embedding Models

Esta técnica (llamada **Mean Pooling**) sigue preservando la representación espacial!

Abre **MUCHAS** puertas en el mundo del NLP y los LLMs esto. Después vemos cómo aprovecharlo

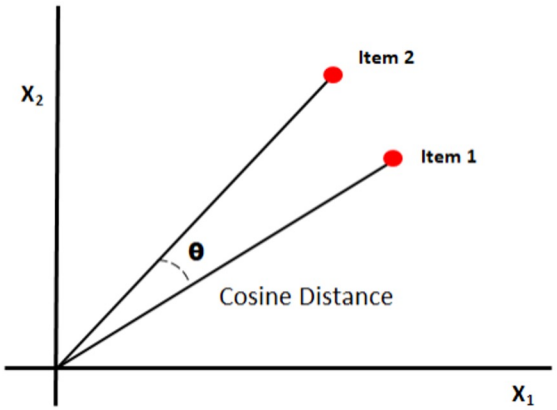


UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Métricas de similitud

Similitud coseno *(Cosine Distance)*

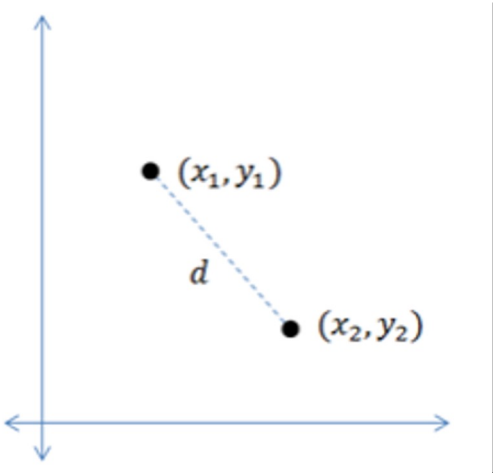
$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$



Aprox iguales si los vectores están normalizados

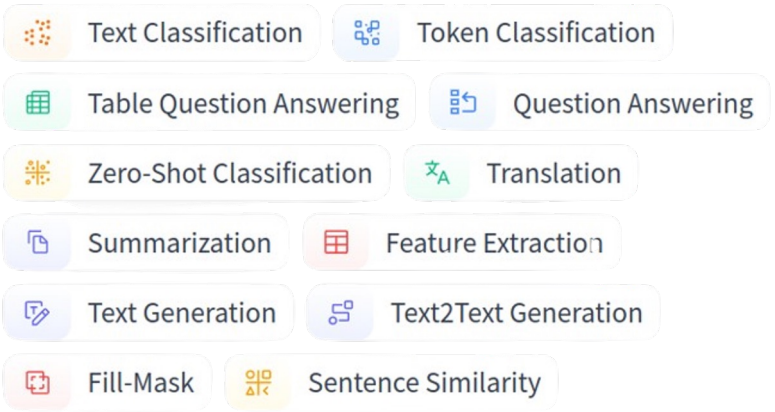
Distancia Euclidiana *(Euclidean Distance)*

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (y_i - x_i)^2}$$



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Con esto podemos resolver todos los problemas?



Respuesta corta... No.

Pero algunos si!

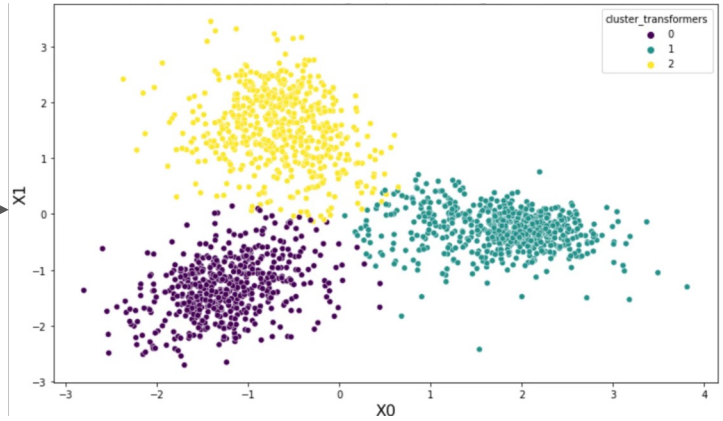
Si pero baja performance capaz?

Clusterización de texto

1536 features

0.1	-0.15	0.22	…	0.12	0.22
0.2	-0.45	0.15	…	0.12	-0.33
-0.1	-0.30	0.33	…	0.14	0.111

PCA
T-SNE
HDBSCAN
+
K Means



Podemos aprender las relaciones entre documentos sin siquiera conocer los documentos!

- Agrupar y estructurar documentos legales?
- Análisis sociales (Sesgos en como *twittean* en función del canal de televisión que siguen).

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Clasificación de texto?

1536 features

0.1	-0.15	0.22	…	0.12	0.22
0.2	-0.45	0.15	…	0.12	-0.33
-0.1	-0.30	0.33	…	0.14	0.111

Etiqueta

True
False
True

Clasificador binario?

- *Identificar noticias falsas*
- *Identificar discursos de odio*
- *Identificar problemas de depresión*

Si pero...

- Cantidad de features es muy grande. *Reducción de dimensionalidad podría ayudar*
- Ejemplos muy variados para que el clasificador pueda realmente aprender. *Me armo más datos? Recopilo más?*

Análisis de sentimiento

1536 features

0.1	-0.15	0.22	…	0.12	0.22
0.2	-0.45	0.15	…	0.12	-0.33
-0.1	-0.30	0.33	…	0.14	0.111

Etiqueta

Feliz
Triste
Neutral

Clasificador multiclase

- **Binario:** Positivo / Negativo.
- **Granular:** Muy Negativo / Negativo / Neutro / Positivo / Muy Positivo.
- **Detección de emociones:** Alegría, Enojo, Tristeza.

Reto principal: La ironía y el sarcasmo. "¡Qué maravilla, mi pedido llegó roto!" (Un modelo básico podría verlo como positivo por la palabra "maravilla").

UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES